Inteligencia Artificial y Neurociencias

Licenciatura en Tecnología Digital, UTDT, 2S-2026

Profesores: Agustín Gravano, Andrés Rieznik
Auxiliares: Gerardo Del Toro, Antonella Giordano Furchi

Fundamentos de LLMs #1

Bibliografía: Jurafsky & Martin, SLP, capítulo 7, “Transformers and Pretraining”

Repaso preliminar

Softmax: Mapea un vector $z \in \mathbb{R}^k$ (llamados *logits*) a una distribución de probabilidades.

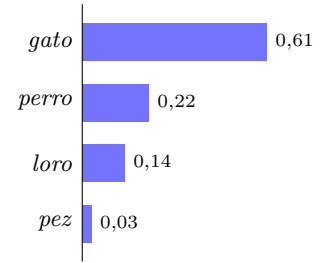
$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{k} e^{z_j}} \qquad 1 \leq i \leq k$$

Por ejemplo, si los *logits* son puntajes crudos asignados a cada palabra de un vocabulario:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{|\text{Vocab}|} e^{z_j}}$$

Ejemplo: Vocab = {*gato*, *perro*, *loro*, *pez*}

	logit z_i	e^{z_i}	prob.
gato	2	7,39	0,61
perro	1	2,72	0,22
loro	0,5	1,65	0,14
pez	−1	0,37	0,03
		Suma:	1,00



Tokenización: ¿cuál es la unidad básica?

- Palabras completas: Poco eficiente. ¿Qué hacemos con las palabras desconocidas? ×
- Caracteres: Se pierde totalmente el significado. ×
- **Fragmentos** *(subwords)*: Se aproxima a la morfología. ✓

  Ejemplo: antiuberización ⟶ `anti` · `uber` · `ización`

  BPE (Byte-Pair Encoding): Algoritmo muy usado para construir el vocabulario de fragmentos, basado en frecuencias.

  > BPE trabaja con textos en UTF-8. Empieza con un vocabulario base con los 256 valores posibles de un byte. Se cuentan los pares de símbolos más frecuentes en un corpus y se fusionan iterativamente en un nuevo token (por ejemplo `t+h` → `th`, luego `th+e` → `the`), hasta alcanzar el tamaño de vocabulario deseado (ej: 100k tokens en GPT-4).

  Código, ejemplos, etc.: `https://huggingface.co/learn/llm-course/en/chapter6/5`

Embeddings

Para cada token id obtenemos su embedding seleccionando su fila correspondiente en la matriz de embeddings:

$$E \in \mathbb{R}^{|\text{Vocab}| \times d}$$

donde:

- $|\text{Vocab}|$: tamaño del vocabulario
- d: dimensión del embedding (768 en GPT-1 $\rightarrow$ 12 288 en GPT-3)

La matriz E se entrena junto al resto de los pesos de la red, punta a punta.

Al token embedding se le suma información posicional: un vector que codifica el lugar del token en la secuencia (hay varias formas de hacerlo).

Resultado: el embedding del i-ésimo token de entrada $x_i \in \mathbb{R}^d$:

$$x_i = \text{token_embedding} + \text{positional_embedding}$$

Atención: intuición central

La fuente tiene características clásicas, con una cresta decorativa en la parte superior de las mayúsculas.

- Los embeddings estáticos no codifican bien el significado de las palabras, porque depende del contexto.
- Al leer la oración de izquierda a derecha, vamos refinando nuestra interpretación de las palabras a medida que tenemos más información del contexto.

Otro ejemplo:

La grúa llevó el auto al taller.

La atención es un mecanismo que pondera y combina las representaciones de tokens en el contexto (*grúa*, *auto*, etc.) para enriquecer la representación de un token dado (*taller*).

Así, *taller* deja de ser un espacio de artesanías o un curso, y pasa a ser un taller mecánico.

Atención: primera versión simplificada

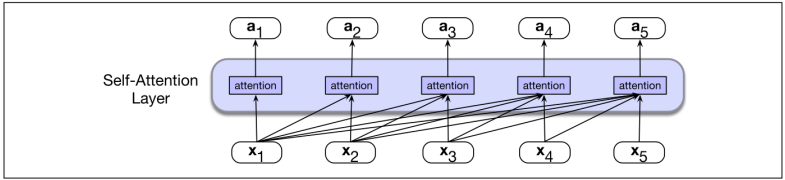


Figure 7.4 Information flow in causal self-attention. When processing each input $\mathbf{x}_i$, the model attends to all the inputs up to, and including $\mathbf{x}_i$.

Primera versión: $$a_i = \sum_{j \le i} \alpha_{ij}\, x_j, \quad \text{con } x_i,\, a_i \in \mathbb{R}^d \quad \text{(vectores fila)}$$

Para los α_{ij} podemos usar una métrica de similitud como el producto escalar y luego normalizar con softmax:

$$\text{score}(x_i, x_j) = x_i \cdot x_j$$
$$\alpha_{ij} = \text{softmax}\left(\text{score}(x_i, x_j)\right) \qquad \forall\, j \le i$$

Ejemplo: $a_3 = \alpha_{31}\, x_1 + \alpha_{32}\, x_2 + \alpha_{33}\, x_3, \quad$ con $\alpha_{ij} \in [0,1] \;\wedge\; \alpha_{31} + \alpha_{32} + \alpha_{33} = 1$

Atención: consultas, claves y valores

Ahora saquemos las simplificaciones. Proyectamos cada input embedding a subespacios con tres roles distintos:

$$\text{Query:} \quad q_i = x_i\, W^Q \quad \longrightarrow \quad \text{¿qué busca } x_i\text{?}$$

$$\text{Key:} \quad k_j = x_j\, W^K \quad \longrightarrow \quad \text{¿qué ofrece } x_j \text{ para ser encontrado?}$$

$$\text{Value:} \quad v_j = x_j\, W^V \quad \longrightarrow \quad \text{¿qué información aporta } x_j \text{ si es elegido?}$$

Entonces:

$$\text{score}(x_i, x_j) = \frac{q_i \cdot k_j}{\sqrt{d_k}} \quad \rightarrow \text{scaling}$$

$$\alpha_{ij} = \text{softmax}\big(\text{score}(x_i, x_j)\big) \quad \forall\, j \le i$$

$$a_i = \sum_{j \le i} \alpha_{ij}\, v_j \cdot W^O$$

Dimensiones:

$$x_i, a_i \in \mathbb{R}^d \quad \text{vectores fila}$$

$$W^Q, W^K \in \mathbb{R}^{d \times d_k}$$

$$q_i, k_j, v_j \in \mathbb{R}^{d_k} \quad \text{vectores fila}$$

$$W^V \in \mathbb{R}^{d \times d_v}$$

$$W^O \in \mathbb{R}^{d_v \times d} \quad \text{reshaping}$$

Transformer original (Vaswani et al., 2017): $d = 512$, $d_k = d_v = 64$

GPT-3: $d = 12\,288$, $d_k = d_v = 128$

Self-attention

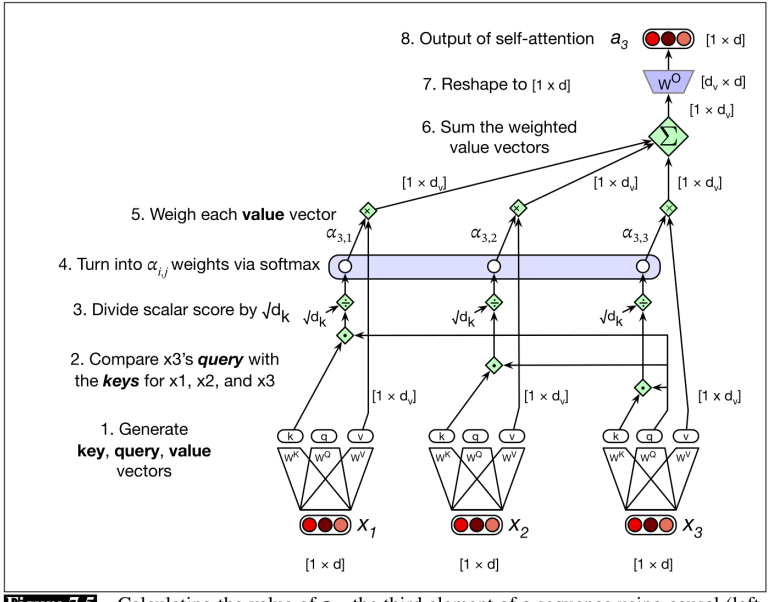


Figure 7.5 Calculating the value of $\mathbf{a}_3$, the third element of a sequence using causal (left-to-right) self-attention.

Multi-head attention: A cabezas de atención en paralelo

Transf: $A = 8$ / 16
GPT-3: $A = 96$
Llama 3.1: $A = 128$

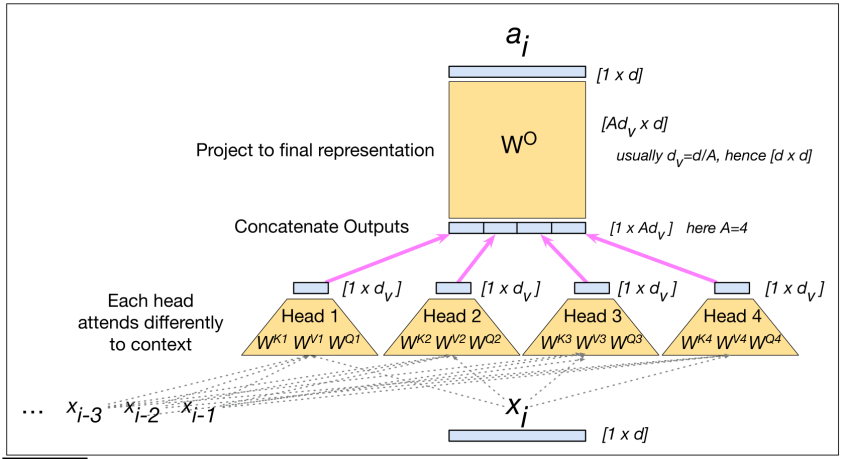


Figure 7.6 The multi-head attention computation for input $\mathbf{x}_i$, producing output $\mathbf{a}_i$. A multi-head attention layer has A heads, each with its own query, key, and value weight matrices. In this figure, we show $A = 4$, a smaller value than is usually used, just to fit on the page. The outputs from each of the heads are of shape $[1 \times d_v]$ and are concatenated and then projected into a different space by the $\mathbf{W_O}$ matrix. Usually the dimensionality d_v of the heads is set so that $d_v = d/A$, with the result that $\mathbf{W_O}$ is a square matrix of shape $[Ad_v \times d] = [d \times d]$. The result is projected to d, producing an output of the same size as the input.

Transformer Block

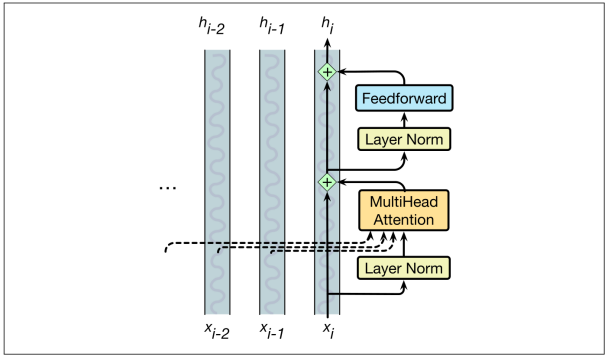


Figure 7.7 The architecture of a transformer block showing the **residual stream**, showing how most information flows up through the residual stream, and only the attention module is sensitive to information from other streams at prior token positions. In this figure and throughout the chapter, we use the **prenorm** version of the architecture, in which the layer norms happen before the attention and feedforward layers rather than after.

Los bloques Transformer se apilan.

Cada bloque combina:

- Layer Norm
- Multi-head attention
- Feed forward
- Conexiones residuales

Transformers: 6 bloques apilados
GPT-3: 96 bloques
Llama 3.1: 126 bloques

Transformer Block: Layer Norm y Feedforward

LayerNorm$(x) = \gamma \cdot \frac{x-\mu}{\sigma} + \beta$, donde μ y σ son la media y stdev de las componentes de x.

- Lleva el vector x a $\mu = 0$, $\sigma = 1$
- Normalización independiente para cada vector; busca estabilizar el aprendizaje
- γ, β: vectores de parámetros, se aprenden (uno por capa)

Feedforward$(x) = W_2 \cdot \text{ReLU}(W_1 \cdot x + b_1) + b_2$

- Red *fully-connected* de 2 capas (i.e., 1 capa oculta) de dimensión d_{ff}.
- Usualmente $d_{ff} = 4 \cdot d$
- Interpretación / intuición: Si la atención *mezcla* información contextual entre tokens, la FF *almacena* o *codifica* conocimiento sobre los tokens.

Residual stream: La atención y la FF agregan (suman) información al vector, que funciona como un *canal* que atraviesa todo el modelo. Cada componente lee de él y escribe sobre él.

Forma matricial

$X \in \mathbb{R}^{N \times d}$: embeddings de los N tokens de la secuencia de entrada.
($N = 512$ en GPT-1, …, 128 000 en GPT-4o)

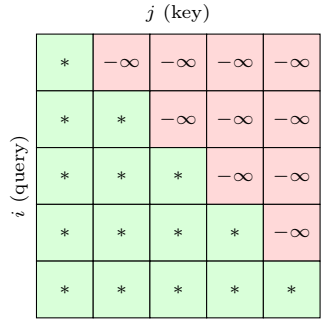
$$
\begin{aligned}
Q &= XW^Q && \text{(queries)} \\
K &= XW^K && \text{(keys)} \\
V &= XW^V && \text{(values)}
\end{aligned}
$$

$$
\begin{aligned}
\text{head} &= \text{softmax}\left(\text{mask}\left(\frac{QK^\top}{\sqrt{d_k}}\right)\right)V \\
A &= \text{head} \cdot W^O
\end{aligned}
$$

Máscara causal

La máscara **oculta la información futura**: pone $-\infty$ en las posiciones $j > i$, y el softmax les asigna probabilidad 0.

$$\text{mask}(S)_{ij} = \begin{cases} S_{ij} & j \le i \\ -\infty & j > i \end{cases}$$



Con $-\infty$ en las posiciones futuras, el softmax les asigna $e^{-\infty} = 0$: peso de atención 0 para el futuro, y cada fila sigue sumando 1.

Language Model Head

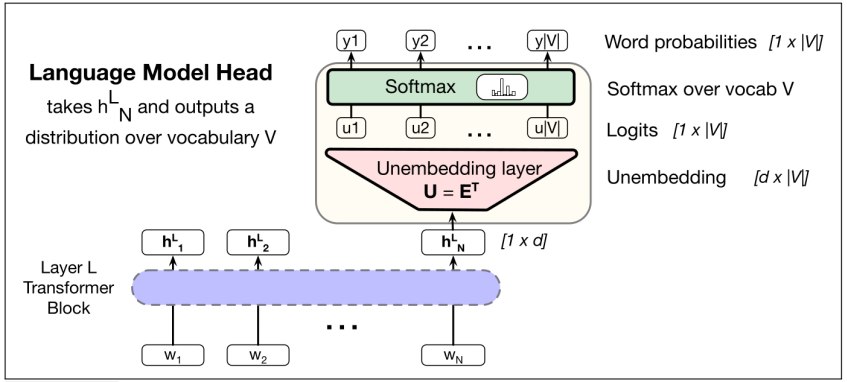


Figure 7.15 The language modeling head: the circuit at the top of a transformer that maps from the output embedding for token N from the last transformer layer ($\mathbf{h}_N^L$) to a probability distribution over words in the vocabulary V.

$$P(tres \mid No\ hay\ dos\ sin) = ? \quad \longrightarrow \quad \text{Usamos } h_4^L \text{ para predecir } w_5.$$

Language Model Head

h_N^L codifica información contextual útil para predecir w_{N+1}.

La LM Head convierte h_N^L a una distribución de probabilidad sobre |Vocab| palabras.

$$u = h_N^L \, E^\top \qquad (u \in \mathbb{R}^{|\text{Vocab}|}, \text{ logits})$$
$$y = \text{softmax}(u)$$

Luego se muestrea el siguiente token w_{N+1} a partir de y.

Intuición: El entrenamiento empuja a h_n^L a parecerse al embedding del token que viene a continuación (w_{N+1}).

Decodificación (en inferencia):

- Greedy: el token más probable
- Random / Top-k random: al azar (sobre top k) según la distribución del softmax
- Temperatura $\tau \in \mathbb{R}^+$: $y = \text{softmax}(u/\tau)$. Con $\tau < 1$, la distribución tiende a greedy; con $\tau > 1$, tiende al azar.

Transformer LLM

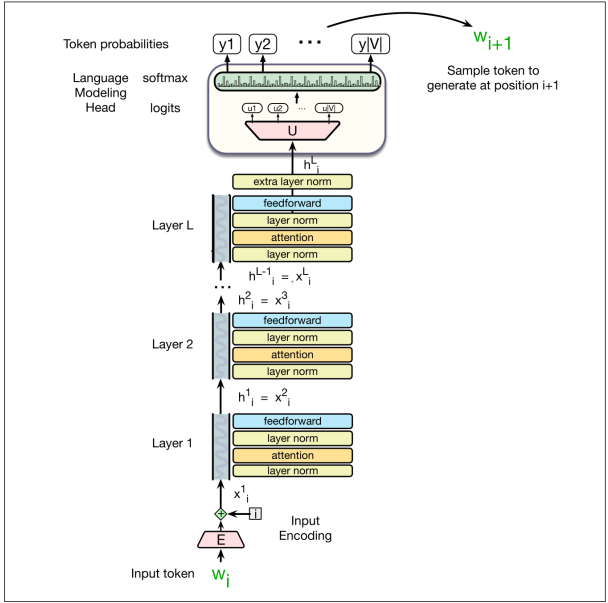


Figure 7.16 A transformer language model (decoder-only), stacking transformer blocks and mapping from an input token w_i to a predicted next token w_{i+1}.

Entrenamiento: pérdida de entropía cruzada

$$\mathcal{L}_{CE}(\hat{y}, y) = -\sum_{w \in |\text{Vocab}|} y[w] \cdot \log \hat{y}[w] \qquad y[w] = \begin{cases} 1 & \text{si } w = \text{próxima palabra } w_{N+1} \\ 0 & \text{para las demás palabras} \end{cases}$$

Por lo tanto, $\mathcal{L}_{CE}(\hat{y}, y) = -\log \hat{y}[w_{N+1}] = -\log P(w_{N+1} \mid w_1, \ldots, w_N)$

Intuición: El modelo recibe *"no hay dos sin"* y tiene que asignarle probabilidad alta a *"tres"*. Si le asigna probabilidad 0,9, la pérdida será baja; si le asigna 0,1, será alta. La pérdida es simplemente $-\log$ de esa probabilidad.

Teacher forcing: Durante el entrenamiento, el modelo recibe como entrada la secuencia correcta completa (N tokens) y predice cada token siguiente en paralelo, produciendo N pares (predicción, token correcto) y N gradientes en una sola pasada. (¡Muy eficiente!)

Así llegamos a la definición completa de pérdida: para cada par, se calcula la cross-entropy entre la predicción y el token correcto:

$$\mathcal{L}_{CE}(\hat{y}, y) = -\frac{1}{N}\sum_{t=1}^{N} \log \hat{y}[w_t] \; = -\frac{1}{N}\sum_{t=1}^{N} \log P(w_t \mid w_1, \ldots, w_{t-1})$$